Lab Four – Programming in R

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.

In this lab, we will build a mealkit recipe generator. I created a small website of about 40 recipes. We will scrape recipes from that website, randomly select three meals, and print a grocery list with recipe cards.

1 Preliminaries

1.1 Part a

Below, I create a numeric vector filled with random observations. Ask for the 3rd item. **Note**: The `set.seed(7272)` portion ensures we all get the same answer.

```r
set.seed(7272) # sets the randomization seed for replication
x <- sample(x=1:10,          # sample from 1, 2, 3, ..., 10
            size=10,         # sample of 10
            replace = TRUE)  # allowed to sample the same item multiple times
x[3] #asking for the 3rd index
```

```
[1] 5
```

1.2 Part b

Below, I create a data frame filled with random observations.

```r
set.seed(7272) # sets the randomization seed for replication
df <- data.frame(x1 = sample(x=1:10,          # sample from 1, 2, 3, ..., 10
                             size=10,         # sample of 10
                             replace = TRUE), # allowed to sample the same item multiple times
                 x2 = sample(x=1:10,          # sample from 1, 2, 3, ..., 10
                             size=10,         # sample of 10
                             replace = TRUE), # allowed to sample the same item multiple times
                 x3 = sample(x=1:10,          # sample from 1, 2, 3, ..., 10
                             size=10,         # sample of 10
                             replace = TRUE)  # allowed to sample the same item multiple times,
)
```

1.3 Part b

Use the `head(...)` function to peek at the data frame.

```r
head(df) #shows first few data in the dataset
```

```
  x1 x2 x3
1  8  1  8
2  3  7  2
3  5  3  3
4  3  6  2
5  2  2  3
6  3  8 10
```

1.4 Part c

Ask for the column `x1`.

```
df$x1
```

```
 [1] 8 3 5 3 2 3 1 2 6 2
```

1.5 Part d

Ask for the fifth row of the data frame.

```
df[5,]
```

```
  x1 x2 x3
5  2  2  3
```

1.6 Part e

Ask for the the value of `x1` in the fifth row of the data frame.

```
df$x1[5]
```

```
[1] 2
```

1.7 Part f

Ask for the the value of in the third column and the fifth row of the data frame.

```
df[5,3]
```

```
[1] 3
```

1.8 Part g

Create a sequence from 1 to 10 by 2 and use it to print the odd rows of the data frame.

```
odd.rows = seq(from=1, to=10, by=2) #using sequential vector
df[odd.rows,] #using odd.rows in place of vector row number
```

```
  x1 x2 x3
1  8  1  8
3  5  3  3
5  2  2  3
7  1 10  1
9  6  2 10
```

1.9 Part h

Below, I create an empty column called `x12`. Fill in the details of the `for(...)` loop to ensure `x12` is the product of `x1` and `x2`.

```
n <- nrow(df)                # How many rows to we have to fill?
df$x12 <- rep(NA, nrow(df))   # Create an empty column for x12

for(i in 1:nrow(df)){
  df$x12[i]=df[i,1]*df[i,2]
}
#for every given row in columns 1 and 2, multiply them.
df[,"x12"]
```

```
 [1]  8 21 15 18  4 24 10 12 12  8
```

1.10 Part i

Write a function called `calculate.score(...)` that takes three arguments representing `x1`, `x2`, and `x3`) and returns a single value based on the formula:

$$Score = (x_1 \times 2) + x_2 - x_3$$

Use your function to create a new column `total.score` in our data frame `df`.

```
x1 = df[,1]
x2 = df[,2]
x3 = df[,3]
calculate.score = function(x1, x2, x3){(2*x1)+x2-x3}
#makes a function that returns the value
df = data.frame(x1, x2, x3,
                total.score = calculate.score(x1,x2,x3))
#includes total.score in the data frame
df
```

```
   x1 x2 x3 total.score
1   8  1  8           9
2   3  7  2          11
3   5  3  3          10
4   3  6  2          10
5   2  2  3           3
6   3  8 10           4
7   1 10  1          11
8   2  6  1           9
9   6  2 10           4
10  2  4  1           7
```

1.11 Part j

Create a function called `evaluate.row(...)` that takes one argument and returns "low" when the argument is less than 4, "mid" when the argument is between 4 and 7 (inclusive), and "high" when the arguement is 8 or larger. Then, use `sapply(...)` to apply it to column `x1`.

```
evaluate.row = function(i){
                        if(i<4){
                         "low"
                        }
                        else if(i>=4 & i<=7){
                         "mid"
                        }
                        else{
                        "high"
                        }
}
#if i is less than 4, return "low,"
#if i is greater than 4 and if i is less than or equal to 7, return "mid,"
# if i is any other number, return "high"

sapply(x1, evaluate.row)
```

```
 [1] "high" "low"  "mid"  "low"  "low"  "low"  "low"  "low"  "mid"  "low"
```

1.12 Part k

Did we need to use loops or functions in (h.)-(j.)? That is, can we use vectorization to attain the same results in 1 line each? Where it is possible, write the line of code. Where it is not, explain why.

I could use a vectorization in part h by filling a manual vector with the product of the data frame's columns x1 and x2.

```
x12 = c(df[,2]*df[,1])
x12
```

```
 [1]  8 21 15 18  4 24 10 12 12  8
```

In part i, I could have also used a manual vector in place of a function, and I would have made calculcated.score = total.score.

```
calculated.score = c((2*x1)+x2-x3)
calculated.score
```

```
 [1]  9 11 10 10  3  4 11  9  4  7
```

In part j, I could have used vectorization with nested if else statements.

2 Complete Tasks for One Recipe

2.1 Part a

Install and load the `rvest` package (Wickham 2025).

```
#install.packages("rvest")
library("rvest")
```

2.2 Part b

Load the html of the `website/KimchiGrilledCheese.html` using the `read_html()` function and save the result to an object called `recipe.item`. We will use the Kimchi Grilled Cheese recipe as our prototype and extend this workflow to all recipes, so try to be as general as possible. If you do look at it, you'll notice it contains the html we saw in the developer tools in class. **Hint**: You can use `read_html(...)` like `read.csv(...)`. Don't forget you can use the documentation to help use it.

```
recipe.item = read_html("website/KimchiGrilledCheese.html")
 #makes the webpage a digestable object for later code.
```

2.3 Part c

Open the html file in a web browser and open developer tools. In chrome-based browsers, you can do this by pressing the three verticle dots in the upper-right corner, clicking "more tools", then "developer tools".

Find the name of the Ingredients section of the website and pull the HTML of the function using `html_element(...)` and save the results to an object called `ingredients.section`. **Hint**: You can ask for elements by id using a preceeding "#". See `?html_element(...)` for a helpful example.

```
ingredients.section = html_element(recipe.item, "#ingredients")
# this pulls the specific section involving ingredients
```

2.4 Part d

Now, we want to obtain all of the itemized items. Find the element type the individual ingredients and pull all of them from `ingredients.section` using `html_elements(...)` (note the added s) and save the results to an object called `ingredients`. **Hint**: You can ask for elements by type by simply specifying the tag (e.g., "p" for paragraph). See '?html_element for a helpful example.

```
ingredients = html_elements(ingredients.section, "li")
#This creates the object that will list out the ingredients from the section
```

2.5 Part e

Similar to Part c. Find the name of the Instructions section of the website and pull the HTML of the function using `html_element(...)` and save the results to an object called `instructions.section`.

```
instructions.section = html_element(recipe.item, "#instructions")
# this pulls the specific section involving instructions
```

2.6 Part f

Now, we want to obtain all of the enumerated items. Find the element type the individual instructions and pull all of them from `instructions.section` using `html_elements(...)` (note the added s) and save the results to an object called `instructions`.

```
instructions = html_elements(instructions.section, "li")
#This creates the object that will list out the instructions from the section
```

2.7 Part g

Find the class of the recipe image and pull the HTML using `html_element(...)` and save the results to an object called `image.element`. **Hint**: You can ask for elements by class using a preceeding ".". See `?html_element(...)` for a helpful example. Further, note that HTML elements may have more than one class (separated by a space). When that is the case, you need to choose one.

```
image.element = html_element(recipe.item,".figure-img")
# this will pull the specific image from the website into a digestible object
```

2.8 Part h

Use the `html_attr(...)` function to pull the source link ("src"). Then, use `paste(...)` to prepend the source link with "website/" so we have the full link. **Note**: This would be like adding "https://www.website.com" to get the absolute link.

```
image.url=paste("website/",html_attr(image.element, "src"), sep="")
# this will paste the specific image from the object
```

2.9 Part i

Now that we have all the things we need, let's try to print the recipe. Below, I have written code to print from the objects. One by one, remove `#| eval: false` from the YAML header and add `#| echo: false` and `#| results: 'asis'`, and test. Let me know if you're stuck!

Image

Figure 1: Kimchi Grilled Cheese

Ingredients

- 1/8 tsp chili flakes
- 4 garlic cloves
- 5 oz spinach
- 1 tsp gochujang
- 3 tsp mayonnaise
- 2/3 cup kimchi
- 4 slices of bread
- 2/3 cup cheddar cheese
- 4 tablespoons everything seasoning.

Instructions

1. In a frying pan, heat 2 tablespoons olive oil with chili flakes.
2. Add spinach and cook until wilted.
3. Mix gochujang and mayonnaise in a small bowl and spread on one side of each slice of bread. Sprinkle with everything seasoning and press it into the bread.
4. Build sandwiches by layering spinach, kimchi, and cheese between two slices of bread.
5. Fry in butter until golden brown on both sides.

3 Complete a Full Menu!

Open `Menu.qmd` and add code to complete the following.

1. Randomly select three dinner recipes and one breakfast recipe at random. **Hint:** Use the `sample(...)` function.
2. Pull the image, ingredients, and instructions for each selected recipe.
3. Combine all of the ingredients into a grocery list on the first page.
4. Print the image, ingredients, and instructions for each receipe on the subsequent pages.

When you render this document, no code should be visible. Instead, you should see a five-page document as described above.

4 Describe your work!

4.1 Why a fake website?

Read https://www.scrapingbee.com/blog/is-web-scraping-legal/. Originally, I conceived doing this with recipes from Purple Carrot (my favorite subscription service). Being a large company, their terms of service are *very* long and precluded us from copying their recipes. We also checked a few smaller recipe websites we like and even they had terms of service that restricted automated collection of data.

Solution

We had to use a fake website due to the ambigious legality associated with scraping. While the act itself is not illegal, there are many factors that can strip scraping of its legality. A major component is that many of these recipe websites incorporate some level of paywall, copyright protections, and lengthy terms of service.

4.2 Conditionals

Did you use conditional statements in your code? If yes, how. If not, are there places you could have used them but did something else?

I did not use conditionals in my code, but I could have used conditionals to differentiate between the different recipes instead of using multiple different objects, but this seemed fairly inefficient when I could just use a general solution for one recipe and apply it to multiple recipes. Addtionally, this would be a bit unconventional considering that different conditions recieve the same general treatment.

4.3 Loops

Did you use loops in your code? If yes, how. If not, are there places you could have used them but did something else?

I did not use loops in my code. I could have used loops to reproduce the general solution across different recipes, but I just opted to do it manually with copy/paste.

4.4 Functions

Did you use functions in your code? If yes, how. If not, are there places you could have used them but did something else?

I did not use functions in my code, but I could have made the code for ingredients, instructions, and images different functions for the sake of brevity and cleaner code.

5 Citations

I used the tidyverse package (Wickham et al. 2019) and (R Core Team 2025) as my coding program, and Rvest (Wickham 2025).

References

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. https://www.R-project.org/.

Wickham, Hadley. 2025. *Rvest: Easily Harvest (Scrape) Web Pages*. https://doi.org/10.32614/CRAN.package.rvest.

Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemund, et al. 2019. "Welcome to the tidyverse." *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.